# TTB Label Verifier

Checking alcohol labels against their applications: approach, design decisions, and trade-offs

**Matthew Nolan** · June 2026

Live demo: [ttb-label-verifier-matthew-nolan-s-projects.vercel.app](ttb-label-verifier-matthew-nolan-s-projects.vercel.app)

Source: [github.com/Mjnolan91/ttb-label-verifier](github.com/Mjnolan91/ttb-label-verifier) (the README covers setup, architecture, and deployment)

## 1. What this is

A compliance agent uploads photos of an alcohol label. A vision model reads them, and the tool checks what is printed against what the application claims: the brand name, the alcohol content within the legal tolerance for its class, the net contents on an authorized standard of fill, the government warning, verbatim and correctly formatted, and the rest of the nine-field application (class and type designation, producer name and address, country of origin), each compared only when the application supplies it. The verdict is Approve, Needs review, or Reject, with a card for every field explaining how it was compared. The same engine runs a batch worklist for the 200-to-300-label dumps the discovery notes describe, and every read exports to JSON and RFC-4180 CSV carrying the same field set, the verdict, the reviewer's decision, and the partial-read fact. Both exports derive from one field catalog, so they cannot drift, and the saved record is the audit record.

Three numbers I kept in front of me while building. The test suite (800 tests) runs offline in seconds with no API keys, so every behavior described here is reproducible on a laptop. The evaluation harness gates every commit on approve precision over 27 labeled cases with a hard floor of 98 percent, because a false approval is the one error I decided this tool would not ship. And the deployed demo measured a median of 3.1 seconds end to end against the five-second line the Deputy Director drew. Section 7 has the full measurement story, including the configuration mistake I found later and what it changed.

## 2. The choices I made on purpose

Four decisions run through everything below, each with its reasoning and its cost.

### AI extracts, code decides

Vision models transcribe printed text reliably, but their judgments are uneven and cannot be audited, and a compliance verdict has to be auditable. So the model gets exactly one job: transcribe what is printed into eighteen structured fields, each with a confidence, plus four tri-state warning-format flags (true, false, or could not tell). Everything that decides is plain, unit-tested TypeScript: pure comparators for the field-by-field match, a completeness matrix encoding what each beverage class must carry, and the statutory constants (the 27 CFR 16.21 warning text, the tolerance bands, the standards of fill) in one hand-verified module. I do not let tests retune that module. If a test disagrees with the statute, the test is wrong.

The payoff is that when the tool rejects a label, the reason is a rule with a CFR citation, not a model's opinion.

## Failing gracefully

Jenny's line in the discovery notes, that badly photographed labels "should fail gracefully," shaped this design more than any other sentence in the brief. I read it as an error model, not a feature request about blurry photos. The version I held myself to: the tool is allowed to say it is not sure; it is never allowed to be silently wrong. Concretely:

- An unreadable photo gets a re-upload prompt, never a fabricated verdict.
- If one of a product's images cannot be read (a dense back label is the slowest thing in the pipeline), the response names which image and why, both screens warn, and the verdict is capped at Needs review no matter how clean the surviving fields look: a dropped back label must not pass itself off as a clean front-only read.
- A field whose value matches the application but was read at low confidence shows "Match · confirm photo" rather than a green pass. The values agreeing is not the same thing as the photo being trustworthy.
- The bold-prefix check on the warning is tri-state. "Could not be verified" routes to review: the tool never claims a violation it cannot see, and never passes one it cannot rule out.
- A container size that is not on the authorized list routes to review rather than fail, because TTB amends the authorized-size lists (the January 2025 final rule added 28 sizes across spirits and wine), so an unlisted size is more likely a rule change than a violation, and the right reader for that is a person.
- In batch, a busy service retries with jittered backoff while an adaptive gate halves the pool's concurrency on a transient failure and creeps it back up, TCP style; the row narrates each attempt, and when retries run out it says so and offers a Retry.
- A batch row that settles cleanly but missing a mandatory element gets one background second look a few seconds later: a focused re-read of exactly the missing fields on the strong model, with a prompt built for those entries. A find surfaces at review confidence for a person to confirm, because a value the first read missed and a retry found is unstable evidence, not settled fact.
- Review state cannot go stale. Every flagged field carries confirm and flag controls, and a recorded decision drafts the applicant email; any re-read or application edit invalidates all of it and the row returns to Undecided, because a stale Approved must never force-pass a recomputed verdict.

Under all of it is one asymmetry: an unnecessary review costs an agent a few minutes, while a false approval costs trust in the whole tool. The error budget is spent on the cheap error, and CI enforces that posture: the build fails if approve precision drops below 98 percent on the labeled cases.

## The application is the headline, not the AI

The brief's core check is label versus application, so that comparison leads the screen; the AI's field table is supporting material. The model's reading pre-fills every application input as a gray suggestion, accepted with Tab or one click, and that is what removes the manual data entry the agents complained about. But a suggestion only becomes part of the application when a person accepts it, so the model never approves its own work. The verdict waits until every field TTB

requires for the beverage type is supplied, and the required set changes with the type: a malt beverage does not need an alcohol statement, spirits do.

### Offline by default

The default vision provider is an offline mock that keys off the image filename to return fixture reads, which means the whole suite, the eval, and CI run deterministically with zero keys and no network. The mock is not a parallel test world: it implements the same provider interface as the real providers, so the 800 tests exercise the entire production pipeline (merge, vote, comparators, verdict) with only the one nondeterministic call swapped out. The cost: out of the box the app only recognizes the bundled samples, and the screen says so plainly. Reading arbitrary photos takes one environment variable and a key; the deployed demo runs that way.

## 3. How the pipeline earns a verdict

Six interchangeable providers sit behind one VisionProvider interface: the offline mock, OpenAI, Gemini, Azure OpenAI, Azure Document Intelligence OCR, and an ensemble mode that runs both Azure providers and routes any disagreement between them to review. I kept the real ones boring: strict structured outputs (the JSON schema carries the per-field instructions, so the prompt and the field definitions cannot drift), bounded retries that honor Retry-After headers (OpenAI's millisecond retry-after-ms takes precedence over the coarse seconds value), and self-limiting request timeouts so one straggler cannot spend the whole latency budget.

A product's images (front, back, and neck if there is one) ride a single request per sample as position-labeled parts, so the model reads the panels with cross-panel context: brand on the front, warning on the back, one record out. It is also the cheaper shape: a two-image product pays per-sample requests, not images times samples. I considered the obvious alternative, stitching the photos into one composite, and rejected it on the resolution math: the vision APIs cap total image resolution, so stitching halves each label's pixels exactly where the fine print lives, and the published measurement agrees (MMNeedle, NAACL 2025: stitched sub-images collapse fine-grained retrieval versus separate image parts). Providers that cannot take a joint read (the offline mock, the Azure OCR path) read each image separately and deterministic code merges the panels; that per-image path is where a single image can drop out, which is why a drop is reported instead of swallowed. If the model reports that two panels contradict each other on a field, that field is pinned into the review band and made ineligible for every automatic recovery below, so the contradiction always lands with a reviewer.

Each product is read N times in parallel (three by default; the recommended live pairing is five samples under an eight-second per-call cap), and a field's confidence is the fraction of samples that agree. I trust that number far more than a model's self-reported confidence. Cosmetic variation clusters as one reading; numeric differences never cluster. Disagreement is tiered: when most samples read a value identically and one sample dropped the field, it lands at 0.65, gated to review but still eligible for the rescue below, because one dropout in a wide vote is usually sampling noise. A genuine split or any numeric disagreement stamps 0.3, deliberately below the rescue band, so a real conflict stays with a human.

One recovery is not a model call at all. The samples sometimes file a printed PRODUCT OF FRANCE under a sibling field, so plain code harvests an origin statement that names a country into the empty origin field, carrying the source read's confidence. It is code over text the model already produced, never another request, and a region like Imported from the Caribbean never qualifies.

Three escalation paths sit alongside the vote, each on its own bounded budget, and each allowed to clear a false alarm but never to flip a conflict. A strong-model judge handles the bold-prefix check, the one input to a hard fail that must come from the model's eyes rather than the transcript; it is sampled up to three times per image and majority-voted, it starts concurrently with extraction so its latency hides behind the read, and even then a lone not-bold from one source never fails a label, because the extraction read and the judge must agree. A rescue spends exactly one extra call re-reading only the fields still under the 0.7 trust gate on the strong model: cross-model agreement clears the false alarm, disagreement adopts the stronger reading but keeps the review hold. And when the warning is still missing or unverified after both, a two-call warning-focus pass locates it in any orientation, crops, derotates, and upscales the region (sharp, server side), and re-judges from the zoomed crop. A warning recovered this way lands at review confidence, not a silent pass, because the statutory text sits in every model's training data and a transcription that merely matches it proves nothing by itself. One invariant ties them together: a deliberate review hold is never released by an escalation agreeing with itself; the same strong model re-reading its own words is not independent evidence.

Latency is enforced, not hoped for. The samples run in parallel, so wall-clock is the slowest sample rather than the sum; every model call carries a hard per-call cap (eight seconds in the recommended configuration); and the judge runs concurrently with extraction, so its couple of seconds hide behind the read. Each escalation runs on its own time budget, a rule I learned the hard way: my first version had the rescue share the per-sample cap, which made it a guaranteed dead timeout by the time it fired. The rescue now runs on its own clock.

The last word belongs to deterministic code: the completeness check and the comparators, with the headline verdict taking the worse of the two. The completeness matrix scores every TTB-mandatory element for the class as present, missing, malformed, or unverifiable, and it encodes the per-class nuance that keeps the check honest: malt beverages may omit the alcohol statement, wine at or under 14 percent may print a table wine designation instead of a number, the warning itself is exempt under 0.5 percent ABV, and the sulfite declaration is conditional, because 27 CFR 4.32(e) requires it only at 10 ppm or more and no photo can establish parts per million, so it is surfaced rather than failed. Match is not compliance here. An origin statement that matches the application verbatim but names a region instead of a country still routes to review, because the marking rules require a country. And there is exactly one verdict derivation, shared by the eval harness, the single screen, and the batch worklist, so there is no second copy to drift, and the CI precision floor gates the exact code path the screens render, not a laboratory proxy of it.

## 4. The bundled sample, as an example of the posture

The verify screen's sample product is a real label (Fireball Cinnamon Whisky, from TTB's public COLA registry), a front and back pair placed into the slots with one click. It is an import for a reason: the back's importer line and PRODUCT OF CANADA statement exercise the origin rules end to end, and accepting the AI's suggestions walks the whole nine-field application to an Approve.

Two details of the defect variant are worth explaining. First, the registry artwork is web resolution, and at that resolution the boldness of the warning prefix cannot be verified by any reader, human or model; the system answers "could not be verified" and routes to review. Rather than let the demo pretend otherwise, the sample's warning block is re-rendered at print fidelity, and the two back variants differ only in the statutory variable. Second, my first defect was bold-versus-regular weight, and the live system kept routing it to review instead of rejecting. That is the conservative tri-state doing its job, and it makes a mushy demonstration. So the

bundled defect prints "Government Warning:" in title case, the exact defect Jenny describes rejecting, which is judged from the transcript and fails deterministically. The real product's label is compliant, and every surface that mentions the variant says so.

## 5. What the stakeholders asked for

Each concern from the discovery notes maps to a specific, testable behavior:

| Stakeholder concern | What the app does |
|---|---|
| Sarah: results in about 5 seconds or nobody uses it (the 30 to 40 second scanner was abandoned) | Hard latency budget: parallel reads with a per-call straggler cap, the judge hidden behind extraction wall-clock, browser-side image downscaling. Measured live: p50 3.1s, p95 5.2s, 15 of 15 verdicts correct. (Measured on the original single-image configuration; section 7 has the multi-image correction and the re-measured dense pair.) |
| Sarah: an interface her 73-year-old mother could figure out | One accessibility-first screen (WCAG 2.1 AA: 4.5:1 contrast in both themes, 44px primary targets, full keyboard order, visible focus, live regions). Gray AI suggestions with Tab-to-accept, an Accept-all button, and a ? explainer on every field. |
| Sarah and Janet: importers dump 200 to 300 applications at once | The /batch worklist: drop hundreds of images; fronts and backs pair by filename, an optional CSV supplies the application values, and same-brand rows offer a one-click combine that a person confirms, never a fuzzy auto-merge. Rows stream in continuously with triage chips, attention-first sorting, per-row retry, in-place application editing, and CSV export. |
| Dave: STONE'S THROW vs Stone's Throw is obviously the same thing; you need judgment | Layered brand comparison: case, punctuation, diacritics, and smart quotes normalize to a pass with the reason stated; a symbol-only difference (Smith & Co vs Smith Co) still routes to review because symbols can distinguish registered brands; near-misses get a similarity-scored review, not a hard fail. |
| Jenny: the warning must be exact, word for word, caps and bold; title case gets rejected | Word-for-word comparison against the statutory 27 CFR 16.21 text (kept in one constant, guarded by a verbatim unit test), explicit all-caps and bold prefix checks, and the bundled defect sample reproduces her exact rejection: a title-case, regular-weight prefix. |
| Jenny: labels photographed badly should fail gracefully | Taken as the error model for the whole tool; see section 2. Unreadable photos get a re-upload prompt, dropped images are reported and cap the verdict at review, and uncertainty routes to a person instead of a guess. |
| Marcus: the firewall blocks outbound ML endpoints; we are on Azure | The in-tenant production path is Azure OpenAI plus Azure Document Intelligence behind the same provider interface, so the model calls never leave the tenant. The public demo runs OpenAI; switching is configuration. |

## 6. Tools and stack

| Layer | Choice |
|---|---|
| Application | Next.js 16 (App Router, standalone output), React 19, TypeScript strict, Tailwind CSS 4. Runtime dependencies are next, react, react-dom, and sharp (server-side image cropping for the warning-focus pass): no database, nothing else to operate. |
| AI models | Demo: OpenAI gpt-4.1 extraction with a gpt-5.5 warning judge; the Gemini path keeps the same split (gemini-3.5-flash transcribes, gemini-3.1-pro-preview judges). Production target: Azure OpenAI and Azure AI Document Intelligence. All behind one VisionProvider interface; the offline mock is the default. |
| Testing | Vitest (800 tests, offline), Testing Library for components, a tsx evaluation harness over 27 labeled fixtures acting as a CI gate, and live measurement scripts for deployed latency. |
| CI / hosting | GitHub Actions (typecheck, lint, test, eval, build on every push, fully offline). Vercel hosts the public demo; a multi-stage Dockerfile targets Azure App Service or Container Apps for in-tenant deployment. |

## 7. Measured, not claimed

The standard for this section is simple: a performance or accuracy claim traces to a dated measurement, or it is not in here.

- **Offline gate.** 800 tests in 62 files pass in seconds with no network. The eval harness scores 27 labeled cases (clean labels plus deliberate defects: a title-case warning, an out-of-tolerance ABV, a brand typo, a missing warning, an import without a country statement, an unreadable photo that must never auto-approve) at 100 percent, with the 98 percent approve-precision floor enforced in CI.
- **Live latency.** 15 sequential reads against the deployed demo measured p50 3.1s and p95 5.2s with 15 of 15 verdicts correct; the one 5.2s read was the serverless cold start (June 10). The model split was chosen by A/B measurement, not preference: moving extraction to the strong model doubled the median (p50 2.8s to 6.5s) for identical verdicts, and on the Gemini path the strong model also failed 4 of 9 requests outright on its 25-requests-per-minute preview quota. So on either vendor the fast model transcribes, and the strongest model is spent only on the judgment that can hard-fail a label.
- **A configuration gap the first measurement missed.** That 3.1s configuration (a 7-wide vote under a 5 second per-sample cap) was tuned on single-image labels, and the numbers looked great because I had measured the easy case. A real front-plus-back product later exposed the gap: a dense, warning-bearing back label reads in about 5 seconds, so the cap could starve every sample of the back image, and at the time the pipeline would proceed silently on the front alone. The fix mattered more than the numbers: a dropped image is now reported end to end, a partial read caps the verdict at review, and the recommended pairing is a 5-wide vote with an 8 second cap, re-measured reading the same dense pair completely in 4.9 seconds. The failure and the diagnosis are documented in the repository.
- **The bundled sample, measured after it shipped (June 11; the dated entry is in the README).** Three sequential rounds per product against the deployed demo: the clean pair read 3 of 3 Approve at 6.6 to 8.7 seconds; the defect pair read 3 of 3 Reject at 12.8 to 14.0 seconds. The defect's extra seconds are the warning-focus pass taking its zoomed

look before committing to a hard fail; the verdict itself is deterministic, because the title-case prefix fails from the transcript.

## 8. What I deliberately did not build

1. Image rectification. Badly photographed labels (angles, glare) get the re-upload prompt rather than deskewing. The brief says handling them is out of scope but failing well is not, and that is where the budget went; the self-consistency vote absorbs ordinary read noise.
2. Physical-measurement rules. Type-size minimums and placement requirements (same field of vision, separate and apart) cannot be measured from extracted text, and I did not want to half-check them. They need layout-preserving OCR, which is the natural next provider capability.
3. Registry-backed checks. Formula approvals and permit matching need TTB records, not the label. Out of scope by design.
4. Authentication and rate limiting on the demo. Upload-size caps and bounded fan-out are enforced in the route, but per-client rate limiting belongs to an API gateway in production, not hand-rolled prototype code.
5. Automatic statute tracking. I verified the statutes by hand instead: the warning text was re-checked against 27 CFR 16.21 in June 2026; the 2025 Surgeon General advisory is a proposal, not law. If the wording ever changes it is a one-constant edit guarded by a test, and a non-blocking note in the UI tracks the pending proposals. The tolerance table is selected per class from the CFR, and the symmetric band is not the whole rule: some classes carry absolute boundaries the band may never cross (the wine 14 percent tax-class line, the malt 0.5 percent floor), enforced separately so a tolerance cannot soften a statutory limit. Two judgment calls inside that table (cider classification, the unknown-class default to the tightest band) are flagged in code for verification before production use.
6. Inflated calibration claims. The Brier and ECE numbers from the offline harness calibrate the pipeline over its fixtures, not a live model, and the documentation says exactly that.

## 9. If this moved toward production

The first steps are deliberately boring: deploy the existing container in-tenant on Azure with provisioned model quota, put the endpoint behind the agency gateway for authentication and rate limiting, and turn on the opt-in escalation pass where accuracy on contested reads is worth the tail latency. After that, layout-preserving OCR to unlock the physical-measurement rules, and a sampled human-audit loop over auto-approvals so the false-approval rate keeps getting measured in the field, not assumed. COLA integration comes last; it is an authorization and procurement question more than a technical one.

The prototype's job was to show that the shape survives measurement: verdicts inside the latency budget, a false-approval floor enforced in CI, and every uncertain read landing with a person. Section 7 is the receipt.